Design of 8-Bit Full Adder

Shivani Mruthyunjaya, Sloan Louden-Robins, Luke Scarpato

***Abstract*—The prompted objective was to design and construct an 8-bit ripple carry adder. To ultimately visualize the CMOS layout within the Microwind software, the construction of the circuit was also completed in LTSpice. The mentioned procedure allowed for verification that the output behavior was as expected. Both simulation and layout results are presented and compared against each other, focusing on the delay and output accuracy. The analysis highlights the trade-offs between simplicity and performance, comparing the 8-bit full adder to a potential Skip Adder implementation. Emphasis on possible improvement for delay through structural and calculative changes were acknowledged.**

***Index Terms*—Full Adder, VLSI, LTSpice, MicroWind, DeMorgan's Theorem**

I. Introduction

THE design and optimization of arithmetic circuits are crucial in VLSI systems and play a large role in ensuring accurate and efficient computation in digital devices. This project demonstrates this in the design and analysis of an 8-bit ripple carry adder using VLSI techniques learned in an undergraduate level introductory course. An 8-bit adder performs binary addition of two 8-bit numbers, producing an 8-bit sum and a carry-out. Composed of cascading 1-bit full adders, it serves as a key component in arithmetic logic units of microprocessors. Each 1-bit adder takes in two input bits and a carry input, generating a sum and carry output for the subsequent stage. While the ripple carry adder is simple in design, its cascading nature introduces delays as the carry propagates through each stage. This makes timing performance a critical factor in its implementation. In this project, we look at how the carry inputs and outputs act like capacitance, affecting how long it takes for signals to move through the full adder. The main job of the 8-bit adder is to compute binary sums accurately and propagate carry bits through its stages. Starting with two binary numbers as inputs, the adder processes each bit starting from the least significant to the most significant bit, summing them while accounting for any carry from the previous stage. The carry propagation creates a delay in ripple carry adders. The speed and accuracy of the adder depend on the design implementation, with factors like gate delays and layout impacting functionality. The ripple carry adder is simpler, however due to its cascading nature it may be slower in computation compared to more advanced designs like look-ahead adders. Eight bit adders are widely used in digital systems that require arithmetic operations, such as microprocessors, digital signal processors, and embedded systems. They are integral in performing calculations, address generation, and other data-processing tasks. In applications requiring speed and efficiency, choosing the right type of 8-bit adder, like a ripple carry or carry-lookahead, can change how well the system works. These adders are also the building blocks for more complex systems like multipliers and accumulators. Finally, the ripple carry adder design is compared to alternative approaches, such as the Skip Adder, to evaluate trade-offs in speed, complexity, and transistor usage. While the ripple carry adder is simpler and resource efficient with its straightforward cascading structure. This simplicity comes at the cost of slower performance due to the inherent carry propagation delay, which increases linearly with the number of bits, meaning the time taken grows proportionally to the number of bits in the system. In contrast, the Skip Adder has a faster performance and reduced delay, which comes at the cost of complexity and higher transistor count. It is significantly faster for large-bit-width additions due to its bits being logarithmically proportional to the propagation delay. The requirements of different tasks, whether they need speed or resources, shape which adder design is best. By understanding the trade-offs involved, designers can choose wisely for their systems, prioritizing either speed or resource efficiency. The above allowed for the project to design a functioning 8-bit full adder with focus upon optimization.

II. Methodology

The natural inclination to proceed with the design for the 8-bit design would be to identify the traditional structure for the 1 bit full adder, and then follow with a sequential process of structure for each full adder feeding into the next consecutively. Yet, this was not the adopted mode of completion for the layout design, as the students placed emphasis on ensuring that optimization was considered during each aspect of the design. It was decided that that 8-bit full adder would be completed in several stages, such that the focus of optimization could be applied as intended. The theoretical analysis to achieve a well optimized circuit was undertaken by initially referencing the traditional 1- bit full adder.

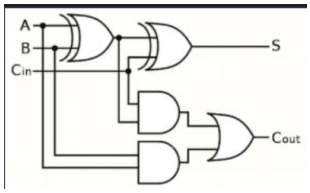

Fig. 1. Traditional 1-Bit Full Adder

Observations of circuit components and behavior yielded the decision to replace several of the components with their NAND equivalent. NAND gates are typically favored during circuit design. The parallel pmos connection within a NAND gate supports efficiency and speed, in contrast to other gates which have configurations that don't support speed. NAND gates are defined by low power consumption and a smaller

delay. Each defining characteristic influenced the students to replace the AND gates and the OR gate with NAND gates (Figure 1). While the replacement seems very direct, this process was only enabled by application of "bubble-pushing" and DeMorgan's Law. Further analysis was completed in an attempt to optimize the delay further. Yet, this approach required equal effort to be achieved at each stage of the circuit, through specially chosen gate sizes. Due to gate sizes being previously defined, this form of optimization was not imposed on the circuit. Satisfaction with the 1-Bit Full Adder prompted the students to consider the layout for the complete 8-bit adder. One approach to the 8 bit adder would be a sequential process where each full adder feeds into the next consecutively. One characteristic that defines a strong overall CMOS layout is being stacked and symmetric.

With this perspective in mind, several different layouts were constructed, employing visual rearrangement and reflection. The chosen 8-bit layout format due to ease of representation and space optimization (Figure 2).

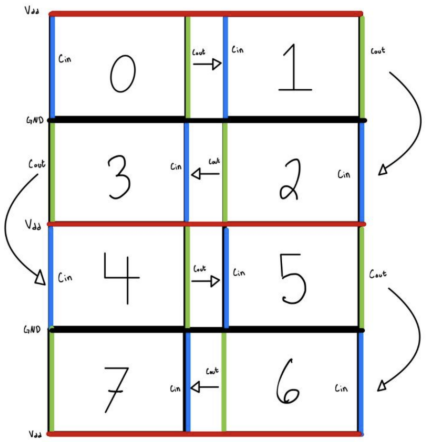

Fig. 2. Visual Representation Of Chosen Layout

After the theoretical understanding of the 1 bit full adder was completed, it was constructed within LTSpice as previously decided (Figure 3).

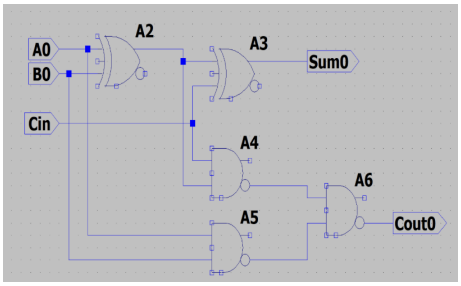

Fig. 3. LTSpice : 1-bit gate design

This process consisted of replicating the intended format, which required the according pulse values into the node positions of A0,B0, and Cin. Then utilizing the chosen layout format, the 1-bit full adder format was entered in the according direction and format, such that the 8-bit full adder could be constructed in LTSpice (Figure 4).

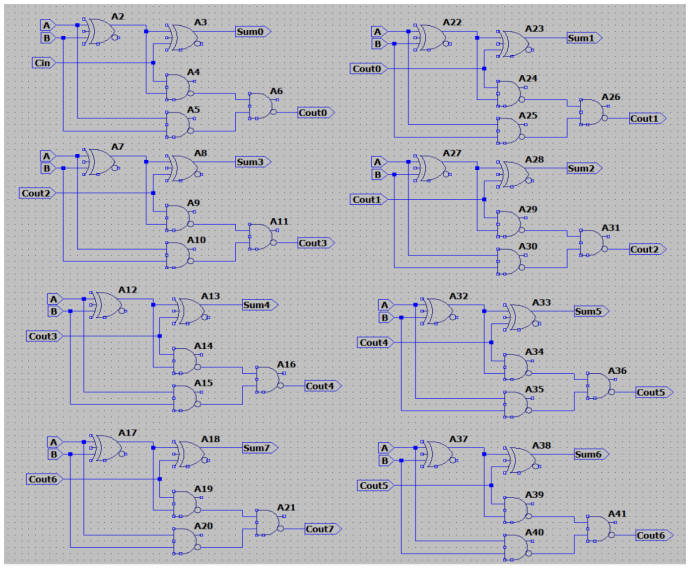

Fig. 4. LTSpice : 8-bit full adder gate design

Verification of the 8-bit full adder gate design prompted the students to produce the transistor diagram for both the 1-bit full adder (Figure 5) and the 8-bit full adder (Figure 6).

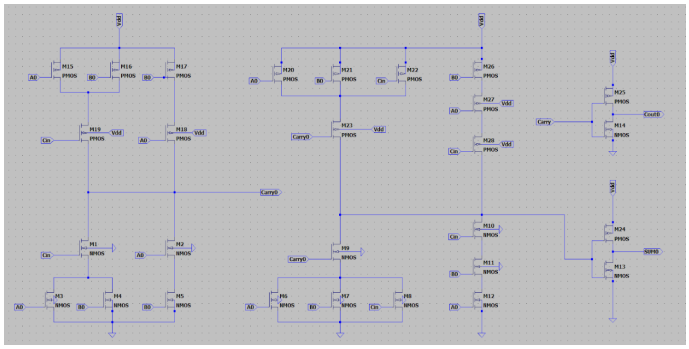

Fig. 5. LTSpice : Transistor Diagram of 1-bit full adder

From the construction of the transistor diagram, the students were able to understand the possible placement of gates such that the correct values could be retrieved from the circuit. While not only allowing visual confirmation of the circuit's creation, the transistor circuit reinforced our understanding of 8-bit adder, and sparked considerations of possible improvements that could be made if alterations could be made for transistor sizing. It was at the creation of this schematic that it was decided that for ease of analysis and testing the inputs to the circuit, the inputs to the circuit would be equal, without any skew between them. Making this decision during the completion of the 1-bit allowed for easy transition to the 8-bit full adder, due to it could be applied for each bit.

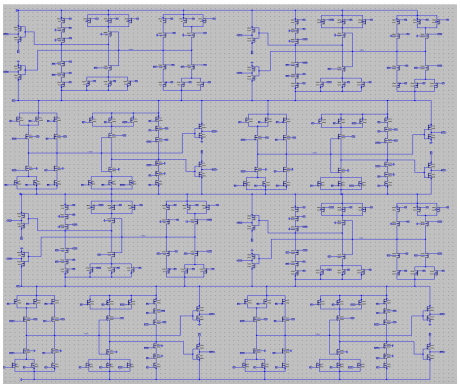

Fig. 6. LTSpice : Transistor Diagram of 8-bit full adder

One interesting aspect of the transistor schematic that came to realization is that our initial decision to focus on spatial efficiency was important and was successfully carried out. In addition, it was realized that this configuration of the 8-bit adder lends itself to scalability very well. In regards to how a stacked configuration, with the reflective property we utilized during theorization, one could seemingly add more bits to the present layout to achieve a large arithmetic addition, without major alterations to the existing design. While well designed for scalability, the ripple carry adder does work especially well for the small scale design, such as in this context. This highlights the modularity of our layout design. One important aspect to consider as well is the inability of LTSpice to consider or take into account signal routing. This is a direct result of LTSpice's execution behavior always presenting the ideal behavior. This process of constructing the transistor diagram was not required, but it supported the technical aspects of the CMOS layout in Microwind. One important aspect that was realized during the construction of the LTSpice Layout was the technical understanding that it does not account for the effect the path length. While the LTSpice schematic provided a reliable representation of the CMOS technical structure, but was not spacially definitive. Mainly due to how all voltage sources were placed externally from the connected transistor circuit. Finally, the student completed the CMOS layout for the 1-bit (Figure 7) and concurrently the 8-bit (Figure 8) CMOS layout within Microwind. Several considerations were taken during the construction of the layout within microwind, including the utilization of "metal 2" to make connections within the layout. To avoid interference with other components within the layout and spacial optimization "metal 2" was utilized.

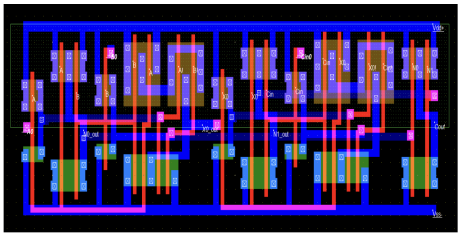

Fig. 7. MicroWind: 1-bit Full Adder

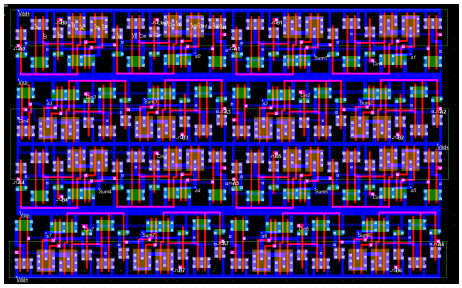

Fig. 8. MicroWind: 8-bit Full Adder

The process of identifying possible connections, maintaining a compressed size and symmetrical format was focused upon during the construction of the layout. Key aspects that were considered were how certain distanced values were connected with an alternate metal type to maintain the stacked shape. The chosen visualization of the theoretical design consisted of not skewed input pulses, such that A0, B0, Cin would have all 1 values or all 0 values at a certain moment of time. Utilization of the LT-Spice and MicroWind software, first a 1-bit full adder was simulated, and the data was recorded. Ensuring the 1-bit function as expected it was then used to implement the 8-bit full adder. The 8-bit full adder is composed of 8 of the 1-bit full adders in a cascading fashion. With some modifications each 1-bit adder is constructed using logic gates optimized with DeMorgan's theorem, reducing complexity and enhancing efficiency. However, due to software limitations and the scope of our knowledge, advanced factors such as gate capacitance, unequal charging and discharging speeds, and signal delays farther from ground were not considered. With these constraints in mind, our design prioritizes optimal speed, symmetry, and efficient use of materials. Procedural application of theoretical concepts in stages allowed the project to achieve an optimized and functional 8-bit full adder design.

III. Results

The construction of the 8-bit full adder within microwind, with the fulfillment of the spacing requirements for each individual component prompted students to evaluate the behavior of the circuit. The accuracy of the 8-bit circuit was verified both through observation and through numerical comparison of delay.

Inputs			Outputs	
A	B	C_{in}	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Fig. 9. 1-bit full adder truth table

For the inputs we tested first at every input set to pulse 1 at the same time (i.e. no skew). This means that every adder

has A and B and the initial Cin inputs are set to high at the same time so going off the truth table for a 1-bit adder means that every adder should have a sum output pulse of 1 and C output of 1. As we see in both the LT spice and Microwind simulations that's exactly what we get. Measurements of the between LT-Spice and MicroWind were compared time delays for each corresponding inputs (i.e. A0 - A7 and B0 - B7) with each its appropriate output (Sum0 -Sum7 and Cout0 - Cout7). Percent error was measured between LT-Spice outputs and Microwind Outputs.

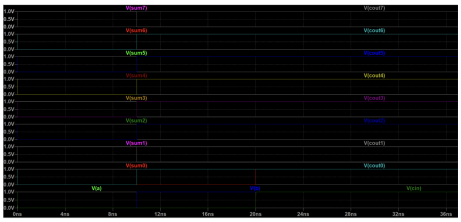

Fig. 10. LT-Spice transistor schematic output measurements

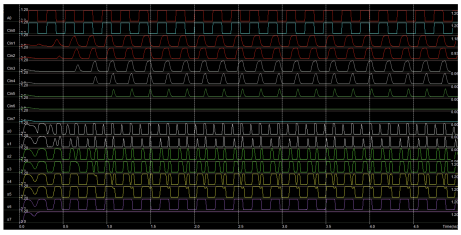

Fig. 11. MicroWind transistor layout output visual

For the LT-spice section we measured an input to output delay of 10ps and in Microwind we estimated an average Propagation delay of 15ps so we determined our percent error to be 50 percent this slight propagation delay in microwind is due to microwind taking into account the delay of the wire and poly, where generally we know that delay of a wire is responsible for 33-40 percent of the total delay. Additionally, our observations highlighted that the voltage across the circuit influenced the carry propagation, with increasing delays in further stages due to cumulative parasitic capacitance. While this is still relative in this context it won't have a noticeable impact on the system's overall behavior because the system is not handling large-scale inputs and the delay is in nanoseconds.

As seen in Fig(10), the pulse pattern demonstrates some delay between inputs and outputs demonstrating the propagation delay in the transistor layout. The reason for these small delays is due to the signal having to propagate through each adder to generate an output. In Fig(11) you can see the signal propagate through each adder but since our input is exactly A = 1 and B =1 for all bits the output is always 1 for each bit. On the SUM7 the output due to the pulse taken over the whole out bit adder we measured the outputs with no carry values; the 8 bit adder's output is 1111 1111 = 255 which is the desired output.

reference input	sum delay		carry delay	
	rise	fall	rise	fall
C0	10ps	3ps	38ps	18ps
C1	13ps	4ps	21ps	16ps
C2	12ps	4ps	21ps	15ps
C3	4ps	12ps	21ps	15ps
C4	4ps	11ps	21ps	-
C5	-	10ps	21ps	-
C6	-	-	-	-
C7	-	-	-	-

Fig. 12. Measurements

In order to take measurements, the input signal causing a change was identified, and then the signal being affected, which was the Sum or Carry, was identified. Then the delay measurement between the input signal and the signal in question was measured and recorded. When referencing the delay table, notice that the prop delay between C5-C7 diminishes to zero, on our first test scenario with no skew and standard pulse width. The reason these values diminish to zero is because the standard pulse width doesn't allow enough time for each stage to discharge and charge. Creating sub-optimal outputs. When we changed the pulse width the system allowed all stages to charge and discharge completely. Compared to Skip Adder the full adder is simpler, requires less transistors and is less prone to error where the skip adder is faster and more efficient. The pros of our layout are that it requires fewer gates, it's simpler and spatially efficient but the system is slower and if one portion of the design fails it will cascade into the other portions of the system.

IV. Conclusion

The creation of a functional and optimized 8-bit adder was the main purpose of the project which was clearly achieved. Yet, several considerations if applied would have reduced the delay within the circuit. One being the utilization of an alternate style of Full Bit Adder, such as a look ahead adder. As this adder reduces the propagation delay by introducing more complex hardware. In addition, the calculation that would support a shorter delay is one that achieves equal effort at each stage. Enabling equal effort at each stage would be the according gate sizes, which would need to be specially chosen. One important aspect to consider is that there is significant accuracy in delay measurements and visual output pulses between the LTSpice layout and the Microwind layout. Despite this, we do see somewhat of a discrepancy in the delay measurement, which was identified as a causation of the Microwind layout having propagation delays, and having certain fall and rise times. In contrast to LTSpice which have immediate, and ideal transitions for outputs. Another aspect that may be partly responsible for the discrepancy is the effect that the path of the carry the bit has on the delay, is not considered in LTSpice, while it was considered within Microwind.